

FACULDADE COTEMIG
Fundamentos Teóricos da Computação

Implementação de Máquinas Abstratas: AFD, Autômato de Pilha e Máquina de Turing

Trabalho Final da disciplina de Fundamentos Teóricos da Computação,
apresentado ao Professor Júlio César da Silva.

Integrantes

Nome	Matrícula
Matheus Felipe de Almeida Mota	72301031
Pedro Simão do Carmo Diniz	72301376
Yuri Leal de Sousa	72301341

Belo Horizonte – MG
2026

Resumo

Este relatório descreve a modelagem e a implementação, em C# (.NET 9), de três máquinas abstratas da teoria da computação: o Autômato Finito Determinístico (AFD), o Autômato de Pilha (AP) com aceitação por pilha vazia e a Máquina de Turing (MT). Cada implementação é uma tradução direta da definição matemática formal. São apresentados os fundamentos teóricos, as decisões de projeto, os resultados dos testes obrigatórios e uma análise comparativa do poder computacional das três máquinas.

Sumário

1	Introdução	3
2	Fundamentação Teórica	3
2.1	Autômato Finito Determinístico (AFD)	3
2.2	Autômato de Pilha (AP)	3
2.3	Máquina de Turing (MT)	4
3	Descrição da Implementação	5
3.1	Parte 1 — AFD	5
3.2	Parte 2 — Autômato de Pilha	6
3.3	Parte 3 — Máquina de Turing	6
4	Resultados e Testes	6
5	Análise Comparativa	7
6	Questões Reflexivas	8
6.1	Parte 1 — AFD	8
6.2	Parte 2 — Autômato de Pilha	8
6.3	Parte 3 — Máquina de Turing	9
7	Conclusão	10
	Referências	10

1 Introdução

A teoria da computação estabelece uma hierarquia de modelos formais capazes de reconhecer classes de linguagens com diferentes poderes expressivos. Este trabalho tem por objetivo modelar e implementar três desses modelos — o Autômato Finito Determinístico, o Autômato de Pilha e a Máquina de Turing — relacionando, em cada caso, a estrutura de dados escolhida com a respectiva definição matemática.

O código foi escrito em C# sobre a plataforma .NET 9 e organizado em três projetos independentes (Parte1, Parte2 e Parte3), cada um separado em camadas de *modelo* (a tupla formal e suas estruturas), *serviço* (simulação e construção das máquinas) e *aplicação* (entrada e saída). O objetivo pedagógico é evidenciar que cada máquina é apenas a expressão computacional de sua definição formal, e não uma solução empírica.

2 Fundamentação Teórica

2.1 Autômato Finito Determinístico (AFD)

Um AFD é uma 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$, em que Q é um conjunto finito de estados, Σ é o alfabeto de entrada, $\delta : Q \times \Sigma \rightarrow Q$ é a função de transição (total), $q_0 \in Q$ é o estado inicial e $F \subseteq Q$ é o conjunto de estados de aceitação. Uma cadeia $w \in \Sigma^*$ é aceita se a computação de M sobre w termina em um estado $q \in F$.

A linguagem-alvo é $L_1 = \{w \in \{a, b\}^* \mid w \text{ termina com } ab\}$, cuja expressão regular equivalente é $(a|b)^*ab$. O AFD construído possui:

$$Q = \{q_0, q_1, q_2\}, \quad \Sigma = \{a, b\}, \quad q_0 = q_0, \quad F = \{q_2\},$$

com a função de transição dada pela Tabela 1.

Tabela 1: Função de transição δ do AFD para L_1 .

δ	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$*q_2$	q_1	q_0

Cálculo manual. Para $w = bab$: $q_0 \xrightarrow{b} q_0 \xrightarrow{a} q_1 \xrightarrow{b} q_2 \in F$, logo ACEITA. Para $w = ba$: $q_0 \xrightarrow{b} q_0 \xrightarrow{a} q_1 \notin F$, logo REJEITA.

2.2 Autômato de Pilha (AP)

Um AP é uma 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, \emptyset)$, em que Γ é o alfabeto da pilha, $Z_0 \in \Gamma$ é o símbolo inicial da pilha e $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$. Adotamos a aceitação **por pilha**

vazia: a cadeia w é aceita se existe uma computação que consome toda a entrada e termina com a pilha vazia.

A linguagem-alvo é $L_2 = \{a^n b^n \mid n \geq 1\}$. Empilha-se um símbolo A para cada a lido e desempilha-se um A para cada b ; ao final, um ε -movimento remove Z_0 , esvaziando a pilha.

Cálculo manual 1 (aceitação). Para $w = aabb$, partindo de $(q_0, aabb, Z)$:

$$(q_0, aabb, Z) \vdash (q_0, abb, AZ) \vdash (q_0, bb, AAZ) \vdash (q_1, b, AZ) \vdash (q_1, \varepsilon, Z) \vdash (q_1, \varepsilon, \varepsilon),$$

pilha vazia com entrada consumida: ACEITA.

Cálculo manual 2 (rejeição). Para $w = aab$, partindo de (q_0, aab, Z) :

$$(q_0, aab, Z) \vdash (q_0, ab, AZ) \vdash (q_0, b, AAZ) \vdash (q_1, \varepsilon, AZ).$$

A entrada foi consumida, mas a pilha ainda contém AZ (sobrou um A sem b correspondente) e não há transição $\delta(q_1, \varepsilon, A)$ definida: a computação trava sem esvaziar a pilha, logo REJEITA.

Como desafio, implementa-se também $L_3 = \{w \in \{a, b\}^* \mid w = w^R, |w| \geq 1\}$ (palíndromos), que exige *não determinismo*: o autômato empilha a primeira metade, “adivinha” o ponto médio (por ε -movimento, no caso par, ou lendo o símbolo central, no caso ímpar) e compara a segunda metade desempilhando.

Diferença de complexidade entre L_2 e L_3 . Embora ambas sejam livres de contexto, as duas linguagens ocupam posições distintas dentro dessa classe. L_2 é uma linguagem livre de contexto *determinística* (DCFL): em cada configuração há no máximo uma transição aplicável, pois a própria estrutura da cadeia (a 's seguidos de b 's) indica, pelo símbolo lido, se o autômato deve empilhar ou desempilhar; um AP determinístico (DPDA) a reconhece. Já L_3 é o exemplo clássico de LLC *inerentemente não determinística*: em uma única passada da esquerda para a direita, o autômato não tem como identificar o ponto médio da cadeia, pois essa decisão depende de símbolos que ainda não foram lidos. Demonstra-se formalmente que nenhum DPDA reconhece os palíndromos sobre $\{a, b\}$ [2]: a estratégia obrigatória — empilhar a primeira metade e compará-la com a segunda — exige “adivinhar” onde a primeira metade termina, poder de escolha que um autômato determinístico não possui. Portanto $L_2 \in \text{DCFL}$, enquanto $L_3 \in \text{LLC} \setminus \text{DCFL}$ — e é exatamente por isso que o simulador implementado precisa explorar múltiplas computações alternativas (busca em profundidade com retrocesso) para L_3 , enquanto para L_2 uma única computação determinística basta.

2.3 Máquina de Turing (MT)

Uma MT é uma 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, com $\Sigma \subseteq \Gamma$, $\sqcup \in \Gamma \setminus \Sigma$ e $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$, sendo $q_{\text{accept}} \neq q_{\text{reject}}$.

A linguagem-alvo é $L_4 = \{a^n b^n c^n \mid n \geq 1\}$. A estratégia é de *marcação*: a cada ciclo marca-se um a (para X), um b (para Y) e um c (para Z), retornando à esquerda e repetindo até que não reste nenhum a ; em seguida verifica-se que só restam Y e Z antes do branco.

Cálculo manual 1 (aceitação, esboço) para $w = abc$:

$$q_0 \underline{abc} \vdash X q_1 \underline{bc} \vdash XY q_2 \underline{c} \vdash X q_3 \underline{Z} \vdash \dots \vdash q_{accept},$$

todos os símbolos marcados como X, Y, Z : ACEITA.

Cálculo manual 2 (rejeição) para $w = ab$:

$$q_0 \underline{ab} \vdash X q_1 \underline{b} \vdash XY q_2 \underline{\sqcup}.$$

Em q_2 a máquina procura um c não marcado, mas encontra o branco $\sqcup$; como $\delta(q_2, \sqcup)$ não está definida, a computação para em estado não-aceitador, comportamento equivalente a q_{reject} : REJEITA.

O desafio implementa uma segunda MT que **computa** $f(n) = n + 1$ em unário: a entrada possui n símbolos 1 e, ao final, a fita contém $n + 1$ símbolos 1 (percorre-se a fita à direita até o primeiro branco, que é substituído por 1).

3 Descrição da Implementação

Cada projeto é um aplicativo de console .NET, organizado em Models/, Services/ e Program.cs, seguindo o princípio de responsabilidade única e evitando classes aninhadas.

3.1 Parte 1 — AFD

A classe AutomatoFinitoDeterministico expõe os cinco componentes da tupla com nomes fiéis à definição (Estados, Alfabeto, Transicao, EstadoInicial, EstadosAceitacao). A função δ é, conforme exigido, um Dictionary<(string estado, char simbolo), string>. O método Simular percorre a cadeia símbolo a símbolo, registra o rastro de estados e retorna um ResultadoSimulacao; Aceitar é um atalho booleano. O método ExibirDiagrama imprime a tabela de transições. O desafio é atendido por AfdJsonLoader, que carrega qualquer AFD a partir de afd.json (campos estados, alfabeto, estadoInicial, estadosAceitacao e transicoes), validando o esquema antes da construção: campos obrigatórios presentes, símbolos de exatamente um caractere, transições que referenciam apenas estados de Q e símbolos de Σ , e ausência de transições duplicadas (que violariam o determinismo de δ).

3.2 Parte 2 — Autômato de Pilha

A pilha é uma `Stack<char>` manipulada manualmente. A função δ é um `Dictionary` de `(string estado, char entrada, char topo)` para uma lista de transições, permitindo não determinismo; ϵ -movimentos são representados por `'\0'`. O `SimuladorPilha` explora as computações por busca em profundidade com conjunto de configurações visitadas (para evitar laços) e limite de passos. A cada passo é registrada a configuração instantânea (estado, entrada restante e conteúdo da pilha), exibida tanto para a computação aceitadora quanto para o caminho mais profundo explorado em caso de rejeição.

3.3 Parte 3 — Máquina de Turing

A fita é uma estrutura dinâmica `Dictionary<int, char>`, com o branco representado por `'_'`. A função δ é um `Dictionary<(string estado, char simbolo), (string novoEstado, char novoSimbolo, char direcao)>`. O `SimuladorTuring` mantém um contador e um limite configurável de passos (evitando laços infinitos), que pode ser definido por argumento de linha de comando (ex.: `dotnet run -project Parte3 - 5000`), e exibe, a cada passo, o estado atual, a fita completa com o símbolo sob o cabeçote entre colchetes e a posição do cabeçote. Há estados explícitos `qaccept` e `qreject`; transições indefinidas são tratadas como rejeição.

4 Resultados e Testes

Tabela 2: Resultados da Parte 1 (AFD para L_1).

Cadeia	Resultado	Observação
ab	ACEITA	caso mínimo
aab	ACEITA	termina com “ab” (ver nota)
bab	ACEITA	prefixo irrelevante
ababab	ACEITA	múltiplas ocorrências
ba	REJEITA	termina com “a”
ϵ	REJEITA	cadeia vazia
b	REJEITA	um símbolo

Nota sobre aab. O enunciado sugere REJEITA para aab, mas a cadeia *aab termina* com o sufixo *ab* e, portanto, pertence a L_1 . A implementação segue rigorosamente a definição formal de L_1 (coerente com bab, também aceita), resultando em ACEITA.

Todos os casos obrigatórios foram reproduzidos pela execução dos projetos (`dotnet run -project ParteX`), com saída completa dos rastros, das configurações instantâneas e dos passos da fita.

Tabela 3: Resultados da Parte 2 — $L_2 = \{a^n b^n\}$ e L_3 (palíndromos).

Cadeia	Resultado	Linguagem
ab	ACEITA	$L_2, n = 1$
aabb	ACEITA	$L_2, n = 2$
aaabbb	ACEITA	$L_2, n = 3$
aab	REJEITA	L_2
abb	REJEITA	L_2
ba	REJEITA	L_2
ε	REJEITA	$L_2, n \geq 1$
abab	REJEITA	L_2
a	ACEITA	L_3
aba	ACEITA	L_3
abba	ACEITA	L_3
ab	REJEITA	L_3
aab	REJEITA	L_3

Tabela 4: Resultados da Parte 3 — $L_4 = \{a^n b^n c^n\}$ e $f(n) = n + 1$.

Entrada	Resultado / Saída	Observação
abc	ACEITA	$n = 1$
aabbcc	ACEITA	$n = 2$
aaabbbccc	ACEITA	$n = 3$
aabbcc	REJEITA	quantidades distintas
ab	REJEITA	sem c
abcabc	REJEITA	fora de ordem
ε	REJEITA	$n \geq 1$
1	11	$f(1) = 2$
111	1111	$f(3) = 4$
11111	111111	$f(5) = 6$

5 Análise Comparativa

As três máquinas formam a base da hierarquia de Chomsky. O AFD reconhece exatamente as **linguagens regulares**; sua “memória” é finita (apenas o estado corrente), o que o impede de contar uma quantidade arbitrária de símbolos. O AP acrescenta uma **pilha**, reconhecendo as **linguagens livres de contexto**, como $L_2 = a^n b^n$ e os palíndromos L_3 — impossíveis para um AFD. Entretanto, a pilha (acesso LIFO, único ponto de acesso) ainda é insuficiente para $L_4 = a^n b^n c^n$, que exige duas comparações de contagem independentes. A MT, com fita infinita de leitura e escrita bidirecional, reconhece as **linguagens recursivamente enumeráveis** e ainda *computa funções*, sendo estritamente mais poderosa que os modelos anteriores. Resumidamente:

Regulares $\subsetneq$ Livres de Contexto $\subsetneq$ Recursivamente Enumeráveis.

6 Questões Reflexivas

6.1 Parte 1 — AFD

1. Por que L_1 é regular? Expressão regular equivalente.

L_1 é regular porque pode ser descrita por uma expressão regular e reconhecida por um autômato finito com memória limitada a três estados. Basta acompanhar o sufixo já lido: “nada relevante”, “acabei de ler um a ” e “terminei em ab ”. Como o reconhecimento depende apenas dos dois últimos símbolos, uma quantidade finita de informação é suficiente, o que caracteriza precisamente as linguagens regulares. A expressão regular equivalente é $(a|b)^*ab$: qualquer prefixo sobre $\{a, b\}$ seguido obrigatoriamente do sufixo ab . O teorema de Kleene garante a equivalência entre expressões regulares e autômatos finitos, confirmando a regularidade de L_1 .

2. Descrição formal da 5-tupla e justificativa dos estados.

$M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\})$. O estado q_0 representa “nenhum progresso em direção ao sufixo ab ”; q_1 representa “o último símbolo lido foi a ”, ou seja, um candidato a iniciar ab ; q_2 representa “a cadeia lida até aqui termina em ab ” e é o único estado de aceitação. As transições foram escolhidas para manter essa invariante: ao ler b em q_1 chega-se a q_2 (sufixo ab completo); ao ler a retorna-se a q_1 ; e ao ler b fora de q_1 volta-se a q_0 . Essa correspondência direta entre significado e estado torna a construção verificável.

3. E se δ não fosse total? Tratamento de símbolos inválidos.

Se δ não fosse total, existiriam pares (estado, símbolo) sem transição definida e a computação ficaria “presa”, sem destino. Em um AFD formal, exige-se a totalidade, geralmente introduzindo um estado de erro (“estado morto”) não-aceitador para o qual vão todas as transições ausentes. Na implementação, a ausência de uma transição é tratada explicitamente: ao encontrar um símbolo fora de Σ ou um par sem destino, a simulação interrompe e classifica a cadeia como REJEITA, registrando o motivo (símbolo fora do alfabeto ou transição indefinida). Assim, entradas inválidas não causam exceções e o comportamento é equivalente ao de um estado morto.

6.2 Parte 2 — Autômato de Pilha

1. Por que L_2 não é reconhecível por AFD? (Lema do Bombeamento)

Suponha, por absurdo, que $L_2 = \{a^n b^n\}$ seja regular. Pelo Lema do Bombeamento para linguagens regulares, existe um comprimento de bombeamento p tal que toda cadeia $w \in L_2$ com $|w| \geq p$ pode ser escrita como $w = xyz$, com $|xy| \leq p$, $|y| \geq 1$ e $xy^i z \in L_2$ para todo $i \geq 0$. Tome $w = a^p b^p$. Como $|xy| \leq p$, o trecho y é composto apenas de a ’s, digamos $y = a^k$ com $k \geq 1$. Bombeando com $i = 2$, obtemos $a^{p+k} b^p$, que possui mais a ’s do que b ’s e, portanto, não pertence a L_2 — contradição. Logo L_2 não é regular e não pode ser reconhecida por um AFD, pois exige contar uma quantidade ilimitada de a ’s, o que ultrapassa a memória finita de estados.

2. Aceitação por pilha vazia e por estado final são equivalentes?

Sim, são equivalentes em poder de reconhecimento: ambas reconhecem exatamente a classe das linguagens livres de contexto. A demonstração é construtiva. Dado um AP que aceita por estado final, constrói-se outro que aceita por pilha vazia adicionando um novo símbolo de fundo X_0 abaixo de Z_0 e um estado de “esvaziamento”: sempre que o autômato original atinge um estado final, o novo autômato passa a desempilhar tudo por ε -movimentos até esvaziar a pilha. Reciprocamente, dado um AP que aceita por pilha vazia, adiciona-se um estado final que só é alcançado, via ε -movimento, quando o símbolo de fundo especial fica exposto (indicando pilha logicamente vazia). Como cada modelo pode simular o outro, eles reconhecem a mesma classe de linguagens.

3. Papel do símbolo Z_0 .

Z_0 (representado por Z no código) é o símbolo inicial da pilha, presente antes da leitura de qualquer símbolo. Ele cumpre dois papéis: serve como marcador de fundo (“a pilha está logicamente vazia se só resta Z_0 ”) e permite detectar o momento de finalizar a computação. Em L_2 , os símbolos A são empilhados sobre Z_0 e removidos pelos b ’s; quando todos os A ’s foram retirados, um ε -movimento $\delta(q_1, \varepsilon, Z) = (q_1, \varepsilon)$ remove Z_0 e esvazia a pilha, configurando a aceitação. Sem Z_0 não haveria como distinguir “pilha vazia por sucesso” de “pilha vazia por falta de símbolo a comparar”, nem como acionar com segurança o passo final.

6.3 Parte 3 — Máquina de Turing

1. Por que L_4 não é reconhecível por AP? Que propriedade da MT permite?

$L_4 = \{a^n b^n c^n\}$ não é livre de contexto, como mostra o Lema do Bombeamento para LLCs: para $w = a^p b^p c^p$ e qualquer decomposição $w = uvxyz$ com $|vxy| \leq p$ e $|vy| \geq 1$, o trecho vxy não consegue cobrir os três blocos simultaneamente; ao bombear, ao menos um dos símbolos (a , b ou c) fica em quantidade diferente dos demais, saindo de L_4 . A pilha do AP só permite uma contagem “casada” (LIFO): ao comparar a ’s com b ’s, ela é consumida e não resta informação para comparar com os c ’s. A MT supera essa limitação por possuir uma fita de leitura e escrita com movimento bidirecional: ela pode *marcar* símbolos e percorrer a fita repetidas vezes, mantendo duas contagens independentes — propriedade essencial para reconhecer L_4 .

2. Quantos estados a MT de L_4 usou? Estratégia de marcação.

A MT de L_4 utiliza sete estados: q_0 (procura o próximo a não marcado), q_1 (avança até o primeiro b não marcado), q_2 (avança até o primeiro c não marcado), q_3 (retorna à esquerda até o último X), q_4 (verificação final de que só restam Y e Z), além de q_{accept} e q_{reject} . A estratégia de marcação é: a cada ciclo, troca-se um a por X , o primeiro b por Y e o primeiro c por Z , garantindo que as quantidades sejam consumidas na mesma proporção; depois retorna-se ao início e repete-se. O processo termina quando não há mais a : então q_4 confirma que restam apenas Y ’s e Z ’s antes do branco, aceitando. Qualquer desbalanceamento (um b ou c a mais/menos, ou símbolo fora de ordem) leva a uma transição indefinida e, portanto, à rejeição.

3. Um computador moderno é mais poderoso que uma MT? (Church-Turing)

Não. Pela Tese de Church-Turing, qualquer função efetivamente calculável pode ser computada por uma Máquina de Turing; nenhum dispositivo físico realizável computa algo além do que uma MT computa. Um computador moderno é, na verdade, *menos* poderoso em um aspecto: possui memória finita, enquanto a MT dispõe de fita ilimitada. Assim, o computador real é, no máximo, equivalente a uma MT de fita limitada (um autômato finito muito grande). A vantagem prática dos computadores está na velocidade e na conveniência, não no poder computacional: problemas indecidíveis para a MT, como o Problema da Parada, permanecem indecidíveis para qualquer computador real.

7 Conclusão

A implementação das três máquinas reforçou, na prática, a correspondência entre definição matemática e estrutura de dados: o AFD como um dicionário de transições, o AP como uma pilha explícita com não determinismo e a MT como uma fita dinâmica com cabeçote. Observou-se concretamente a hierarquia de poder computacional — cada modelo reconhece linguagens que o anterior não alcança — e compreendeu-se o papel de elementos como o estado morto, o símbolo de fundo da pilha e a estratégia de marcação na fita. O exercício de traduzir formalismos em código tornou tangíveis conceitos centrais da teoria da computação.

Referências

Referências

- [1] SIPSER, Michael. *Introdução à Teoria da Computação*. 2. ed. São Paulo: Cengage Learning, 2007.
- [2] HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. *Introdução à Teoria de Autômatos, Linguagens e Computação*. 3. ed. Rio de Janeiro: Campus/Elsevier, 2003.
- [3] MENEZES, Paulo Blauth. *Linguagens Formais e Autômatos*. 6. ed. Porto Alegre: Bookman, 2011.
- [4] LEWIS, Harry R.; PAPADIMITRIOU, Christos H. *Elementos de Teoria da Computação*. 2. ed. Porto Alegre: Bookman, 2000.